

Relatório 5: Análise da BHT

João Bruno dos Santos, João Lucas Medina Kormann

Universidade Federal de Santa Catarina

I. INTRODUÇÃO

Este relatório tem o objetivo de apresentar a metodologia aplicada para a resolução dos problemas do Laboratório 5 da disciplina Organização de Computadores I, ministrada pelo professor Marcelo Daniel Berejuck, utilizando a linguagem Assembly para a arquitetura MIPS (Microprocessor without Interlocked Pipelined Stages) e o simulador MARS (MIPS Assembler and Runtime Simulator). O trabalho consiste na adaptação de dois códigos em C para assembly utilizando também a ferramenta BHT para tentar prever o comportamento dos branches dos códigos.

II. IMPLEMENTAÇÃO DO PROGRAMA 1

Para a implementação do pequeno programa, primeiro deve-se levar em consideração o código proposto pelo exercício:

```
#include <stdio.h>

int main() {
    int limit;

    printf("Digite o limite do contador: ");
    scanf("%d", &limit); // leitura do limite

    int count = 0;

    for (int i = 0; i < limit; i++) {
        if (i % 2 == 0) { // apenas números pares
            count++;
            printf("Contador: %d\n", i);
        }
    }

    return 0;
}
```

Figura 1 - Código C

Como podemos observar, trata-se de um simples contador que imprime os valores apenas os valores pares. Separando de forma objetiva, temos como variáveis: *limit*, que será digitado pelo usuário, *count*, valor referente à quantidade de números pares e, por fim, “i”, o contador em si que executa

dentro do loop até que seu valor seja maior que *limit*. Além destas, temos também as strings referentes aos prints que devemos iniciar juntamente no *.data*.

```
.data
scanPrompt: .asciiz "Digite o limite do contador: "
condPrompt: .asciiz "Contador: "
quebraLinha: .asciiz "\n"

limit: .word 0
i: .word 0
count: .word 0
```

Figura 2 - Data imprimePares

Para a execução do código em si, começamos carregando os valores referentes à impressão da frase "Digite o limite do contador: ", seguida do carregamento de \$v0 com 5 para ler o próximo inteiro que por sua vez é salvo em -4(\$fp), local reservado para a variável *limit*.

```
.text

addi $sp, $sp, -8
# -4($fp) -> limit
# -8($fp) -> count
# -12($fp) -> i

#####
#Scan limit#
#####

li $v0, 4
la $a0, scanPrompt
syscall

li $v0, 5
syscall
sw $v0, -4($fp)
```

Figura 3 - Scan limit

Partindo então para o loop, carregamos inicialmente em \$t0 o valor de “i” já calculando também o valor de $i\%2$, comparando este resultado com 0 e, caso seja verdadeiro, partindo para o setor

“cond_true” referente à parte dentro do if do código em C.

```
#####
#For par #
#####
for:
lw    $t0, -12($fp)
li    $s1, 2
div   $t0, $s1
mfhi  $t1

beq   $t1, 0, cond_true
j     isoma
```

Figura 4 - For

De forma simples, dentro da condição verdadeira apenas carregamos o valor de count em \$t0 para adicionar +1 e também aproveitamos para imprimir o valor de “i” assim como pedido.

```
cond_true:
#Carregar/add count
lw    $t0, -8($fp)
addi  $t0, $t0, 1
sw    $t0, -8($fp)

#####
#PrintCount#
#####
li    $v0, 4
la    $a0, condPrompt
syscall

li    $v0, 1
lw    $a0, -12($fp)
syscall

li    $v0, 4
la    $a0, quebraLinha
syscall
```

Figura 5 - Print Count

Por fim, caso a condição seja falsa, ou após ela ser executada, partimos para o setor “isoma” que carrega “i” em \$t0, soma +1 e o salva novamente, comparando com o valor de limite para caso ainda seja menor, retornar ao início do for. Caso o valor de “i” ultrapasse *limit*, o programa então carrega 10 em \$v0 e utiliza um último *syscall* para encerrar.

```
#i++
isoma:
lw    $t0, -12($fp)
addi  $t0, $t0, 1
sw    $t0, -12($fp)
lw    $t2, -4($fp)
blt   $t0, $t2, for

li    $v0, 10
syscall
```

Figura 6 - Soma do i e encerramento

III. ANÁLISE DO PROGRAMA 1

Em geral, por ser bem simples, o programa 1 ocorreu assim como o esperado, sendo apenas um pouco complexo o uso de pilha para reserva de espaço das variáveis porém nada muito dificultoso. Quanto ao uso do BHT, este conseguiu prever muito bem o primeiro branch referente ao final do for, alcançando mais de 80% de precisão com qualquer configuração. Entretanto, para o branch referente ao número ser par ou ímpar, a predição do BHT não funcionou muito bem, alcançando no máximo uma precisão de 50% com 2 bits.

Trazendo alguns exemplos de testes temos:

BHT Simulator, Version 1.0 (Ingo Kofler)

Branch History Table Simulator

of BHT entries: 16 BHT history size: 1 Initial value: NOT TAKE

Instruction	Index	History	Prediction	Correct	Incorrect	Precision
	0	NT	NOT TAKE	0	0	0,00
	1	NT	NOT TAKE	0	0	0,00
	2	NT	NOT TAKE	18	2	90,00
	3	NT	NOT TAKE	0	0	0,00
	4	NT	NOT TAKE	0	0	0,00
	5	NT	NOT TAKE	0	0	0,00
	6	NT	NOT TAKE	0	0	0,00
	7	NT	NOT TAKE	0	0	0,00
	8	NT	NOT TAKE	0	0	0,00
	9	NT	NOT TAKE	0	0	0,00
	10	NT	NOT TAKE	0	0	0,00
	11	NT	NOT TAKE	0	0	0,00
	12	NT	NOT TAKE	0	0	0,00
	13	NT	NOT TAKE	0	20	0,00
	14	NT	NOT TAKE	0	0	0,00
	15	NT	NOT TAKE	0	0	0,00

Figura 7 - Entries = 16 size = 1 NT

Branch History Table Simulator

of BHT entries: 16, BHT history size: 2, Initial value: NOT TAKE

Index	History	Prediction	Correct	Incorrect	Precision
0	NT, NT	NOT TAKE	0	0	0,00
1	NT, NT	NOT TAKE	0	0	0,00
2	T, NT	TAKE	17	3	85,00
3	NT, NT	NOT TAKE	0	0	0,00
4	NT, NT	NOT TAKE	0	0	0,00
5	NT, NT	NOT TAKE	0	0	0,00
6	NT, NT	NOT TAKE	0	0	0,00
7	NT, NT	NOT TAKE	0	0	0,00
8	NT, NT	NOT TAKE	0	0	0,00
9	NT, NT	NOT TAKE	0	0	0,00
10	NT, NT	NOT TAKE	0	0	0,00
11	NT, NT	NOT TAKE	0	0	0,00
12	NT, NT	NOT TAKE	0	0	0,00
13	T, NT	NOT TAKE	10	10	50,00
14	NT, NT	NOT TAKE	0	0	0,00
15	NT, NT	NOT TAKE	0	0	0,00

Figura 8 - Entries = 16 size = 2 NT

Branch History Table Simulator

of BHT entries: 16, BHT history size: 2, Initial value: TAKE

Index	History	Prediction	Correct	Incorrect	Precision
0	T, T	TAKE	0	0	0,00
1	T, T	TAKE	0	0	0,00
2	T, NT	TAKE	19	1	95,00
3	T, T	TAKE	0	0	0,00
4	T, T	TAKE	0	0	0,00
5	T, T	TAKE	0	0	0,00
6	T, T	TAKE	0	0	0,00
7	T, T	TAKE	0	0	0,00
8	T, T	TAKE	0	0	0,00
9	T, T	TAKE	0	0	0,00
10	T, T	TAKE	0	0	0,00
11	T, T	TAKE	0	0	0,00
12	T, T	TAKE	0	0	0,00
13	T, NT	TAKE	10	10	50,00
14	T, T	TAKE	0	0	0,00
15	T, T	TAKE	0	0	0,00

Figura 9 - Entries = 16 size = 2 T

Branch History Table Simulator

of BHT entries: 32, BHT history size: 1, Initial value: TAKE

Index	History	Prediction	Correct	Incorrect	Precision
0	T	TAKE	0	0	0,00
1	T	TAKE	0	0	0,00
2	NT	NOT TAKE	19	1	95,00
3	T	TAKE	0	0	0,00
4	T	TAKE	0	0	0,00
5	T	TAKE	0	0	0,00
6	T	TAKE	0	0	0,00
7	T	TAKE	0	0	0,00
8	T	TAKE	0	0	0,00
9	T	TAKE	0	0	0,00
10	T	TAKE	0	0	0,00
11	T	TAKE	0	0	0,00
12	T	TAKE	0	0	0,00
13	NT	NOT TAKE	1	19	5,00
14	T	TAKE	0	0	0,00
15	T	TAKE	0	0	0,00
16	T	TAKE	0	0	0,00
17	T	TAKE	0	0	0,00
18	T	TAKE	0	0	0,00
19	T	TAKE	0	0	0,00
20	T	TAKE	0	0	0,00
21	T	TAKE	0	0	0,00
22	T	TAKE	0	0	0,00
23	T	TAKE	0	0	0,00
24	T	TAKE	0	0	0,00

Figura 10 - Entries = 16 size = 1 T

Como dito anteriormente, a precisão para o branch de índice 2 foi elevada em todos os casos, devido a ser um branch que demora a mudar de estado, sendo considerado até o momento em que *i* passa o valor de limit. Já para o caso do branch no índice

13, este é alternado entre “tomado” ou “não tomado” sempre, dificultando assim a predição devido à sua passada, obtendo assim sempre resultados iguais a 50% (no caso de 2 bits para o history size) ou menos, chegando até mesmo a errar todas as predições caso iniciado com “NOT TAKE” e apenas 1 bit.

Por fim, como formas de melhorar o desempenho, poderíamos alterar a lógica da condição para o branch, onde no nosso código, caso “== 0”, vai para o setor “cond_true”, seguindo o mesmo padrão do “if” em C, porém, a fim de evitar um pulo a mais que o necessário, poderíamos utilizar a lógica inversa, comparando o valor e caso este fosse diferente de 0 (bne), pularia para o final do bloco for, sendo então desnecessário o uso do jump para “cond_true”.

```
beq    $t1, 0, cond_true    # Poderia ir direto para iSoma
j      isoma                # Seria desnecessario
```

Figura 11 - Exemplo beq utilizado

Como última alteração, poderíamos também carregar menos as variáveis, utilizando mais registradores paralelos e evitando utilizar “store word” nos loops, o que aprimoraria a eficiência geral do código porém garantiria menos a segurança e integridade geral dos dados. Contudo, em geral, não houveram muitos aprimoramentos possíveis devido à simplicidade do problema, sendo os apresentados os únicos realmente impactantes encontrados, seguindo o princípio de fazer o mais fidedigno possível ao código em C.

IV. IMPLEMENTAÇÃO DO PROGRAMA 2

O primeiro passo da tradução do programa foi a alocação do espaço para as variáveis locais ao método *main()*, com disposição conforme mostra a Figura X.

```
addi    $sp, $sp, -20
#-4($fp) -> size
#-8($fp) -> i
#-12($fp) -> key
#-16($fp) -> found
#-20($fp) -> array (pointer)
```

Figura 12 - Alocação de espaço na pilha para as variáveis locais

Depois, o programa recebe o tamanho do vetor do usuário, e aloca o espaço necessário para ele, salvando a referência do vetor em array, tudo com seus devidos prompts.

```
# alocando espaco do vetor
lw    $t0, -4($fp)
sll    $t0, $t0, 2    # size * 4
sub    $sp, $sp, $t0
sw    $sp, -20($fp)    # salva referencia
```

Figura 13 - Alocando vetor de acordo com o valor digitado pelo usuário

O primeiro ponto que requer uma atenção maior é o *for loop* utilizado, que itera *size* vezes para preencher o vetor. Nessa estrutura, antes de realizar as operações do seu contexto, há um *branch* condicional antes de entrar nele, que verifica a condicional ($i < \text{size}$). Caso for falso, termina a execução do loop, caso verdadeiro, segue.

```
for_fill:
# condicao
lw    $t0, -8($fp)
lw    $t1, -4($fp)
bge    $t0, $t1, end_for_fill

# scanf
li    $v0, 5
syscall
lw    $t2, -20($fp)
sll    $t1, $t0, 2    # t1 = i * 4
add    $t1, $t2, $t1    # define o endereco = sp + (i*4)
sw    $v0, 0($t1)

#i++
lw    $t0, -8($fp)
addi    $t0, $t0, 1
sw    $t0, -8($fp)
j    for_fill
end_for_fill:
```

Figura 14 - For loop para preenchimento do vetor

Após essa execução, o programa recebe do usuário o número que deve ser procurado dentro do vetor. Então, outro for loop ocorre para o encontrar, porém, dessa vez também há uma condição if dentro dele.

```
for_busca:
# condicao
lw    $t0, -8($fp)
lw    $t1, -4($fp)
bge    $t0, $t1, end_for_busca

# arr[i]==key
lw    $t2, -20($fp)    #arr*
sll    $t1, $t0, 2    #i*4
add    $t1, $t2, $t1
lw    $t3, 0($t1)    #arr[i]
#compara
lw    $t4, -12($fp)
bne    $t3, $t4, endif_key

# if true
li    $t0, 1
sw    $t0, -16($fp)
j    end_for_busca
endif_key:
lw    $t0, -8($fp)
addi    $t0, $t0, 1
sw    $t0, -8($fp)
j    for_busca
end_for_busca:
```

Figura 15 - For loop para encontrar o número

Por fim, há a estrutura if, else, para a mensagem de número encontrado ou não para o usuário.

```
lw    $t0, -16($fp)
beq    $t0, $zero, else_nfound

#found:
li    $v0, 4
la    $a0, foundMsg
syscall
j    endif_found

else_nfound:
li    $v0, 4
la    $a0, nfoundMsg
syscall

endif_found:
move    $sp, $fp
li    $v0, 10
syscall
```

Figura 16 - Estrutura if else

V. ANÁLISE DO PROGRAMA 2

Antes de realizar as otimizações do programa, foram obtidos os seguintes resultados nos testes, com tamanho de vetor 5 e sempre buscando o último número.

BHT size (bits)	Initial value	Precision (total)
1	NT	82.5%
1	T	70.6%
2	NT	76.5%
2	T	58.9%

Tabela 1 - Resultados com 16 BHT Entries

BHT size (bits)	Initial value	Precision (total)
1	NT	82.5%
1	T	70.6%
2	NT	76.5%
2	T	58.9%

Tabela 2 - Resultados com 32 BHT Entries

BHT size (bits)	Initial value	Precision (total)
1	NT	76.5%
1	T	70.6%
2	NT	76.5%
2	T	64.7%

Tabela 3 - Resultados com 8 BHT Entries

A partir dos resultados, é possível concluir que com o valor inicial not take, temos um desempenho superior. Isso é devido ao padrão do loop for, que a maioria de suas iterações é not take. A predominância da precisão em 1 bit no tamanho da BHT mostra que o programa não segue devidamente um padrão de desvio, já que a inércia dos 2 bits não auxilia no resultado.

Analisando o código, podemos verificar que já ocorre a otimização para o not take, provado pela precisão alta de acertos. Isso ocorre pois os jumps não são considerados na BHT. Para comparação, o código pode assumir um padrão de take, conforme mostrado a seguir.

Para a alteração do código, ambos os for loops foram alterados, de modo que o padrão dentro do loop seja take. Para isso, a condicional que ficava no início do loop vai para fora dele, e o jump no

final vira um *blt* (*branch if less than*), de modo a ser take na maioria das vezes.

```
# condicao
lw    $t0, -8($fp)
lw    $t1, -4($fp)
bge   $t0, $t1, end_for_fill

for_fill:
# scanf
li    $v0, 5
syscall
lw    $t2, -20($fp)
sll   $t1, $t0, 2    # t1 = i * 4
add   $t1, $t2, $t1  # define o endereco = sp + (i*4)
sw    $v0, 0($t1)

#i++
lw    $t0, -8($fp)
addi  $t0, $t0, 1
sw    $t0, -8($fp)
lw    $t1, -4($fp)
blt   $t0, $t1, for_fill

end_for_fill:
```

Figura 17 - Estrutura do for modificado

Com essa alteração, foram obtidos os seguintes resultados para tamanho de vetor 5, buscando sempre o último número e 16 BHT entries.

BHT size (bits)	Initial value	Precision (total)
1	NT	70.6%
1	T	76.5%
2	NT	47.0%
2	T	70.6%

Tabela 4 - Resultados do programa otimizado

Ao realizar a alteração, é visível que o padrão not take é melhor, e o código original já é o mais otimizado.

Matriz linha a linha:

Direct Mapping:

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1208	121	91%
8	8	1329	1236	93	93%
16	8	1329	1266	63	95%
16	16	1329	1231	48	96%
16	32	1329	1320	9	99%

Matriz Coluna a coluna:

Direct Mapping:

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1040	289	78%
8	8	1329	1040	289	78%
16	8	1329	1056	273	79%
16	16	1329	1281	48	96%
16	32	1329	1320	9	99%

Fully Associative:

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1264	65	95%
8	8	1329	1296	33	98%
16	8	1329	1296	33	98%
16	16	1329	1312	17	99%
16	32	1329	1320	9	99%

Fully Associative:

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1072	257	81%
8	8	1329	1072	257	81%
16	8	1329	1072	257	81%
16	16	1329	1072	257	81%
16	32	1329	1320	9	99%

2-way Set Associative:

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1264	65	95%
8	8	1329	1296	33	98%
16	8	1329	1296	33	98%
16	16	1329	1312	17	99%
16	32	1329	1320	9	99%

2-way Set Associative:

No of Blocks	Cache block size (words)	Memory access count	Cache Hit Count	Cache Miss Count	Cache Hit Rate
8	4	1329	1072	257	81%
8	8	1329	1072	257	81%
16	8	1329	1072	257	81%
16	16	1329	1282	96%	96%
16	32	1329	1320	9	99%